Great extension — moving from an **OS-level forensic pipeline** to a full **application/software solution** makes your proof-of-concept far more compelling. Let's expand the design so it's not just a set of scripts but a usable forensic **platform**.

💻 Application-Level Architecture

1. Frontend (User Interface)

- **Web dashboard** (React, Angular, or Vue)
  - Case management (open/close cases, assign investigators).
  - Evidence upload (disk images, logs, memory dumps).
  - Visualization (timelines, anomaly heatmaps, graphs).
- **Features:**
  - Drag-and-drop evidence ingestion.
  - Interactive anomaly explorer.
  - Automated report generation (PDF/HTML).

2. Backend (Orchestration Layer)

- **API Gateway** (FastAPI, Flask, or Node.js).
- **Agentic AI Orchestrator** (LangChain/Semantic Kernel).
  - Calls forensic tools (Autopsy, Volatility, Wireshark).
  - Runs anomaly detection models.
  - Aggregates results into knowledge graph.
- **Modular Tool Adapters:**
  - Each forensic tool wrapped as a service (CLI → API).
  - Easy to plug in new tools later.

3. Data Layer

- **Evidence Storage:** Object storage (Azure Blob, AWS S3, local FS).
- **Graph Database:** Neo4j for evidence linkage.
- **Relational DB:** PostgreSQL/MySQL for case metadata.

4. AI Layer

- **NLP Models:** Document/email analysis.

- **ML Models:** Anomaly detection (Isolation Forest, Autoencoders).
- **Reasoning Agent:** Hypothesis generation, anomaly aggregation.

5. Reporting & Export

- Automated forensic reports (structured templates).
- Export to PDF/HTML for presentation.
- Integration with case management systems.

🔎 Anomaly Aggregation (Application-Level)

- **Scoring Matrix:**
  - File anomalies (weight 3).
  - Memory anomalies (weight 4).
  - Network anomalies (weight 5).
- **Aggregation Engine:**
  - Sum weighted anomalies per entity (user, process, file).
  - Threshold → Flag as "Critical Event."
- **Visualization:**
  - Heatmap of anomalies per timeline.
  - Graph view showing anomaly clusters.

🔧 Extensibility for Other Tools

- Design **plugin architecture**:
  - Each tool = microservice with standard input/output.
  - Register tool in "Tool Registry."
  - AI agent decides which tool to call based on evidence type.
- Examples:
  - Plaso (log2timeline).
  - Sleuth Kit.
  - ElasticSearch for log indexing.

🧮 Five-Day Application Pipeline

Day 1 – Foundation

- Set up backend (FastAPI/Flask).
- Define modular architecture.
- Prepare sample forensic dataset.

Day 2 – Tool Integration

- Wrap Autopsy, Volatility, Wireshark as services.
- Build API endpoints for ingestion + analysis.

Day 3 – AI Agent & Anomaly Detection

- Implement orchestration logic (LangChain/Semantic Kernel).
- Add anomaly scoring + aggregation.

Day 4 – Frontend & Visualization

- Build dashboard (React/Angular).
- Integrate timeline + anomaly heatmap.
- Connect to Neo4j for graph visualization.

Day 5 – Demo & Reporting

- Run full case through pipeline.
- Generate automated forensic report.
- Present dashboard + architecture diagram.

✅ By Day 5, you'll have a **working forensic application** — not just scripts — with a dashboard, anomaly aggregation, extensibility for other tools, and a polished demo-ready solution.

Would you like me to **draft a sample UI wireframe** (showing dashboard layout: case list, anomaly explorer, timeline, report export) so you can visualize how the application would look to users?